

Einsendeaufgabe DEV-E3: Supported AI and GenAI Coding – Distributed Smart Task Manager

Distributed Smart Task Manager

Entwicklung einer einfachen verteilten Webanwendung

1 Einleitung

Im Rahmen der Einsendeaufgabe DEV-E3 wurde der bereits entwickelte Smart Task Manager aus DEV-E2 zu einer einfachen verteilten Anwendung erweitert.

Ziel war es, eine Software zu entwickeln, die aus mehreren eigenständigen Prozessen besteht. Dabei kommuniziert ein Backend-Service über HTTP mit einem separaten Worker-Service. Der Worker übernimmt die Verarbeitung einer Aufgabe und liefert das Ergebnis an das Backend zurück. Das Frontend dient als Benutzeroberfläche und stellt die Ergebnisse der Verarbeitung dar.

Die Entwicklung erfolgte unter Verwendung von Visual Studio Code, Node.js, Express sowie KI-Unterstützung. Alle Entwicklungsschritte wurden dokumentiert und durch Screenshots nachvollziehbar festgehalten.

2 Zielsetzung

Ziel der Aufgabe war die Entwicklung einer einfachen verteilten Softwareanwendung.

Die Anwendung besteht aus mehreren unabhängig laufenden Prozessen, die über HTTP miteinander kommunizieren. Das Backend übernimmt die Rolle einer REST-API und leitet Aufgaben an einen separaten Worker-Service weiter. Der Worker verarbeitet die Aufgabe und liefert eine Empfehlung zurück. Dadurch wird die in der Lehrveranstaltung behandelte verteilte Architektur praktisch umgesetzt.

3 Umsetzung

Die Entwicklung erfolgte schrittweise. Zunächst wurde die Projektstruktur erstellt und anschließend die benötigten Bibliotheken installiert. Danach wurden Backend, Worker und Frontend entwickelt. Abschließend erfolgte die Integration der einzelnen Komponenten zu einer verteilten Anwendung.

3.1 Projektstruktur

Das Projekt wurde in mehrere Verzeichnisse gegliedert. Diese enthalten den Backend-Service, den Worker-Service, das Frontend, die Projektdokumentation, die Testdaten sowie die Screenshots.

Die klare Trennung der Komponenten erleichtert Wartung, Erweiterung und Nachvollziehbarkeit der Anwendung.

Abbildung 1: Entwicklung der modularen Anwendung in Visual Studio Code. Der Projektbaum zeigt die getrennten Module Frontend, Backend und Worker; rechts ist der Quellcode des Backend-Services geöffnet (Screenshot 10).

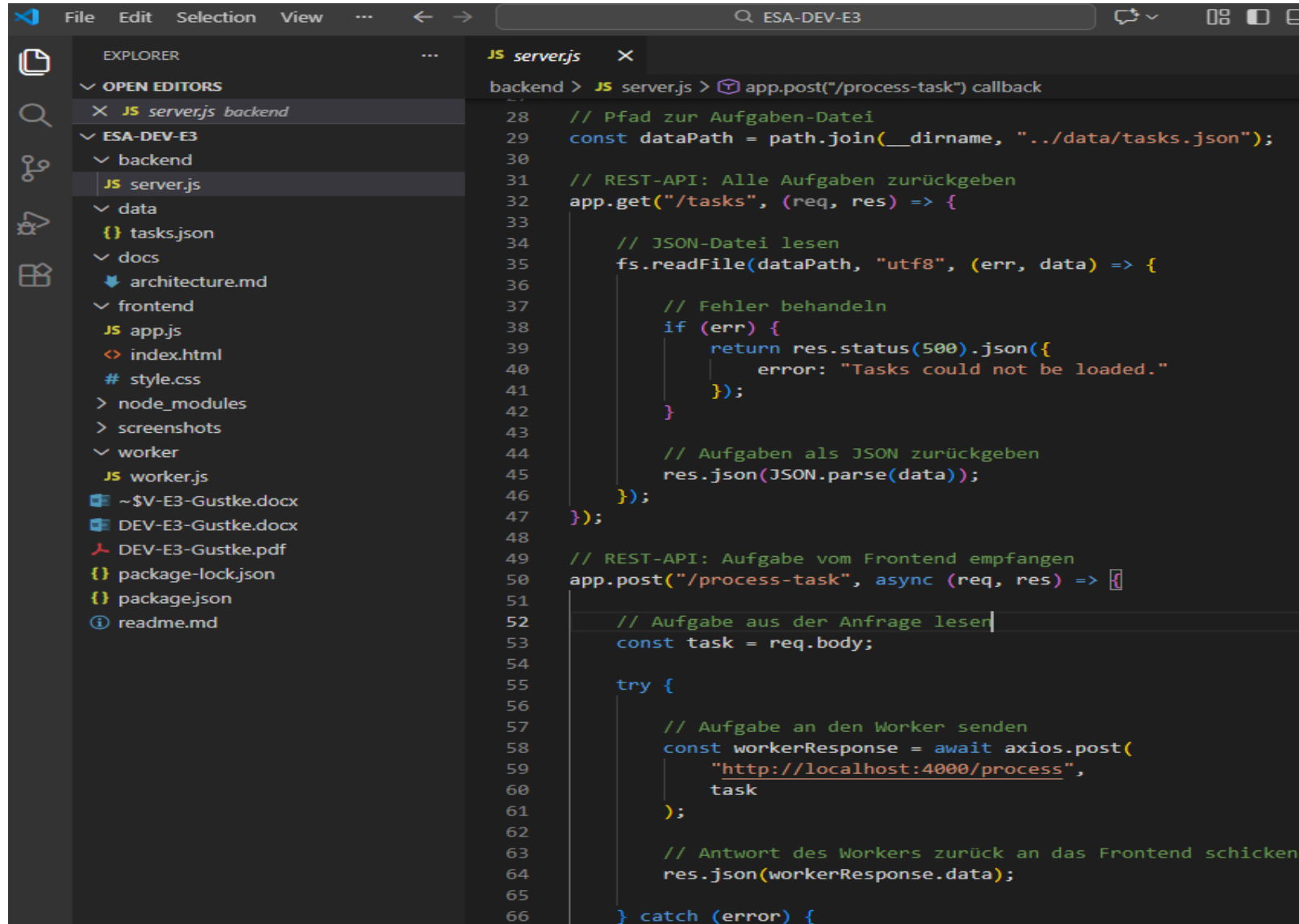
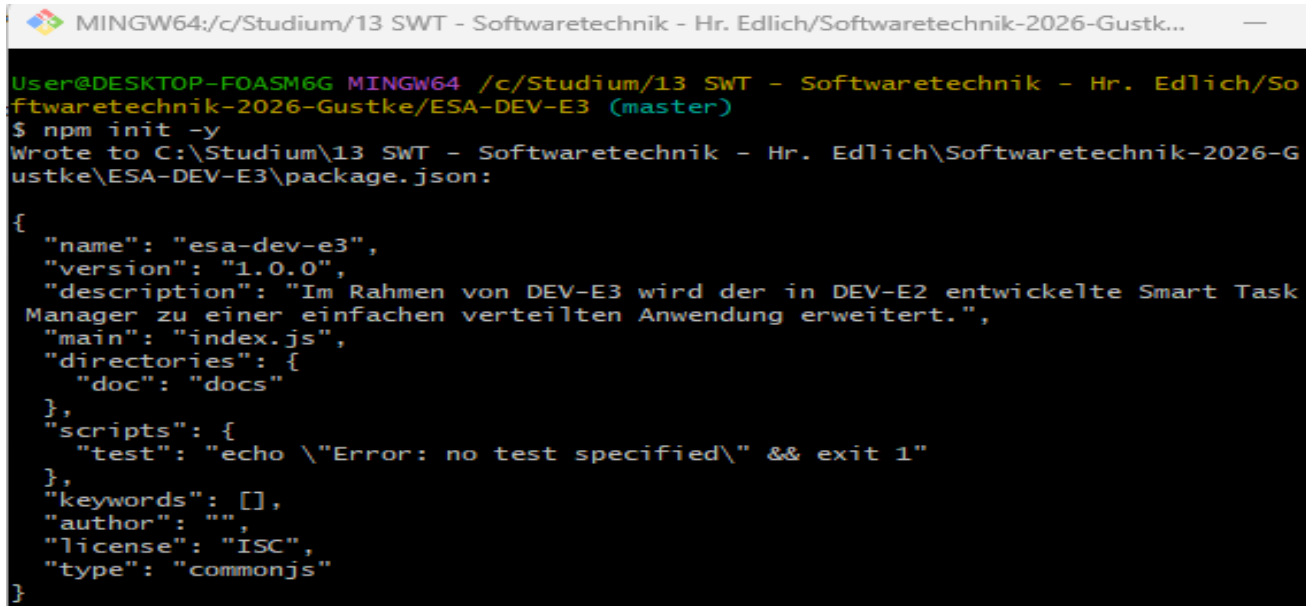


Abbildung 2: Initialisierung des Node.js-Projekts mit npm (Screenshot 01).



```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustk...
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E3 (master)
$ npm init -y
Wrote to C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-DEV-E3\package.json:

{
  "name": "esa-dev-e3",
  "version": "1.0.0",
  "description": "Im Rahmen von DEV-E3 wird der in DEV-E2 entwickelte Smart Task Manager zu einer einfachen verteilten Anwendung erweitert.",
  "main": "index.js",
  "directories": {
    "doc": "docs"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3.2 Installation der Entwicklungsumgebung

Zunächst wurde ein neues Node.js-Projekt erstellt. Anschließend wurden die Bibliotheken Express, CORS und Axios installiert. Express dient zur Bereitstellung der REST-API, CORS ermöglicht die Kommunikation zwischen Frontend und Backend, während Axios die HTTP-Kommunikation zwischen Backend und Worker übernimmt.

Abbildung 3: Installation der Bibliotheken Express und CORS (Screenshot 02).

```
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E3 (master)
$ npm install express cors

added 69 packages, and audited 70 packages in 3s

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E3 (master)
```

Abbildung 4: Installation der HTTP-Bibliothek Axios zur Kommunikation zwischen Backend und Worker (Screenshot 03).

```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E3
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E3 (master)
$ npm install axios

added 13 packages, and audited 83 packages in 2s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

3.3 Verteilte Architektur

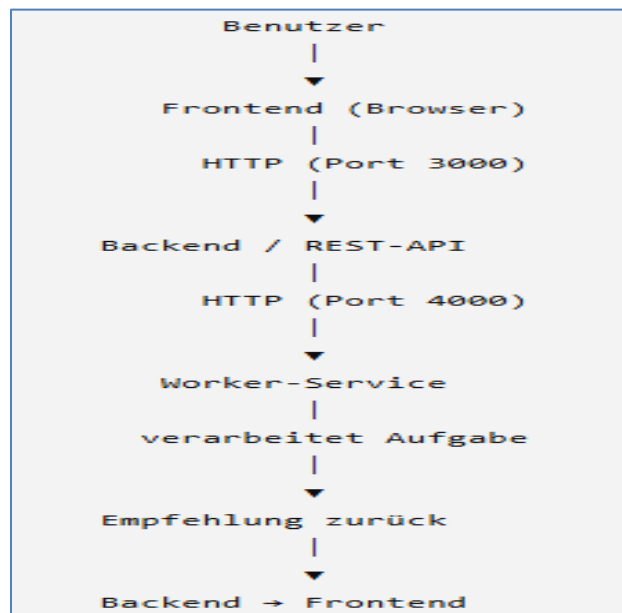
Der wesentliche Unterschied zu DEV-E2 besteht in der Einführung eines separaten Worker-Services.

Während DEV-E2 ausschließlich aus Frontend und Backend bestand, besteht DEV-E3 aus mehreren eigenständig laufenden Prozessen. Das Backend stellt eine REST-API bereit und übernimmt die Kommunikation mit dem Frontend. Für die Verarbeitung einer Aufgabe sendet das Backend die empfangenen Daten per HTTP an den Worker-Service.

Der Worker verarbeitet die Aufgabe unabhängig vom Backend und erzeugt eine Empfehlung anhand der Priorität. Anschließend wird das Ergebnis an das Backend zurückgegeben, das die Antwort an das Frontend weiterleitet.

Durch die Aufteilung in Backend und Worker kommunizieren zwei eigenständige Prozesse miteinander. Damit erfüllt die Anwendung die Anforderungen einer einfachen verteilten Softwarearchitektur.

Abbildung 5: Architektur der verteilten Anwendung. Das Frontend kommuniziert mit dem Backend. Das Backend übergibt Aufgaben per HTTP an den Worker-Service, der sie verarbeitet und das Ergebnis zurückliefert. (Screenshot 09)



3.4 Codeverständnis

Die Anwendung besteht aus drei eigenständigen Komponenten: Frontend, Backend und Worker.

Das Frontend stellt die Benutzeroberfläche bereit. Über die Schaltflächen werden HTTP-Anfragen an das Backend gesendet. Die Antworten werden anschließend im Browser dargestellt.

Das Backend stellt eine REST-API bereit. Es nimmt Anfragen des Frontends entgegen. Beim Laden der Aufgaben liest es die Datei tasks.json und liefert deren Inhalt als JSON zurück. Bei der Verarbeitung einer Aufgabe sendet das Backend die empfangenen Daten per HTTP an den Worker-Service.

Der Worker verarbeitet die empfangene Aufgabe unabhängig vom Backend. Anhand der Priorität erzeugt er eine passende Empfehlung und sendet das Ergebnis als JSON-Antwort an das Backend zurück.

Ich kann den Ablauf der Anwendung vollständig nachvollziehen und die Funktion aller Programmteile erklären. Insbesondere verstehe ich die Kommunikation zwischen Frontend, Backend und Worker sowie die Aufgaben der verwendeten Bibliotheken Express, CORS und Axios.

3.5 Test der Anwendung

Nach der Implementierung wurden alle Komponenten einzeln und anschließend im Gesamtsystem getestet.

Zunächst wurden der Worker-Service und anschließend der Backend-Service gestartet. Beide Prozesse liefen gleichzeitig in getrennten Konsolen auf unterschiedlichen Ports. Anschließend wurde der REST-Endpunkt des Backends überprüft.

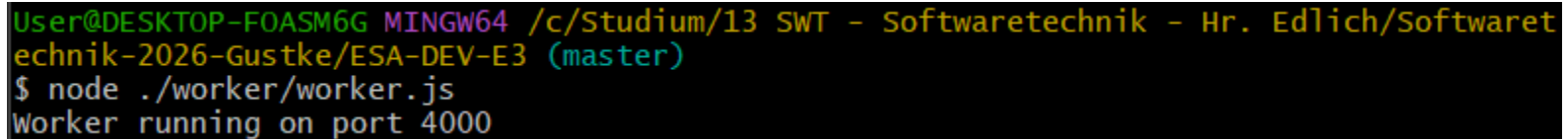
Im nächsten Schritt wurde das Frontend geöffnet. Über die Schaltfläche „Aufgaben laden“ wurden die Aufgaben erfolgreich vom Backend geladen und angezeigt. Danach wurde über die Schaltfläche „Aufgabe verarbeiten“ eine Aufgabe an das Backend gesendet.

Das Backend leitete die Aufgabe per HTTP an den Worker-Service weiter. Der Worker verarbeitete die Priorität der Aufgabe und erzeugte die Empfehlung „Start today.“. Anschließend wurde das Ergebnis an das Backend zurückgegeben und im Frontend angezeigt.

Alle Tests verliefen erfolgreich. Die Kommunikation zwischen Frontend, Backend und Worker konnte vollständig nachgewiesen werden.

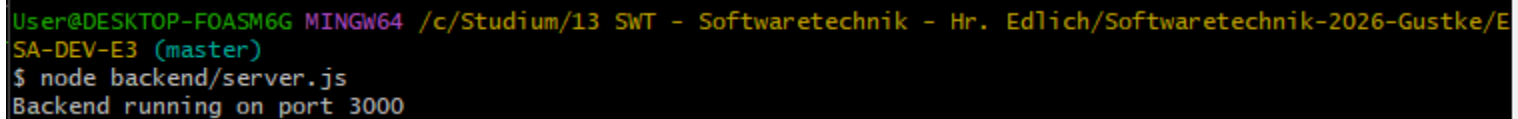
Die wichtigsten Programmteile wurden vollständig nachvollzogen und verstanden. Das Frontend sendet HTTP-Anfragen an das Backend. Das Backend verarbeitet die Anfrage oder leitet sie an den Worker-Service weiter. Der Worker erzeugt anhand der Priorität eine Empfehlung und sendet das Ergebnis an das Backend zurück, das dieses anschließend im Frontend anzeigt.

Abbildung 6: Erfolgreicher Start des Worker-Services auf Port 4000 (Screenshot 04).



```
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaret  
echnik-2026-Gustke/ESA-DEV-E3 (master)  
$ node ./worker/worker.js  
Worker running on port 4000
```

Abbildung 7: Erfolgreicher Start des Backend-Servers auf Port 3000 in einer zweiten Git-Bash-Konsole (Screenshot 05).



```
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/E  
SA-DEV-E3 (master)  
$ node backend/server.js  
Backend running on port 3000
```


Abbildung 8: Erfolgreicher Test des REST-Endpunkts `/tasks`. Das Backend liest die Aufgaben aus der Datei `tasks.json` und liefert sie als JSON-Antwort im Browser aus (Screenshot 06).



Abbildung 9: Backend und Worker laufen gleichzeitig als zwei eigenständige Node.js-Prozesse auf den Ports 3000 und 4000.

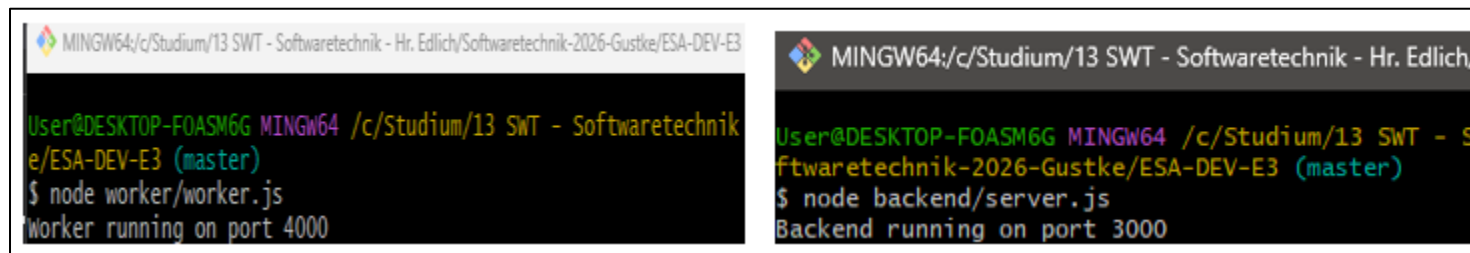


Abbildung 10: Erfolgreiche verteilte Verarbeitung einer Aufgabe. Das Frontend sendet die Aufgabe an das Backend, das sie an den Worker weiterleitet. Der Worker erzeugt die Empfehlung „Start today.“ und sendet das Ergebnis zurück (Screenshot 08).



4 Fazit

Mit DEV-E3 wurde der Smart Task Manager erfolgreich zu einer einfachen verteilten Anwendung erweitert.

Die Anwendung besteht aus einem Frontend, einem Backend und einem eigenständigen Worker-Service. Das Backend kommuniziert über HTTP mit dem Worker und überträgt Aufgaben zur Verarbeitung. Dadurch wurde das in der Lehrveranstaltung behandelte Prinzip verteilter Software praktisch umgesetzt.

Durch den Einsatz von Node.js, Express und Axios konnte eine übersichtliche und leicht verständliche Lösung entwickelt werden. Die Verwendung von KI-Unterstützung beschleunigte die Entwicklung und erleichterte die Dokumentation. Die zentralen technischen Anforderungen an eine modulare und verteilte Anwendung wurden umgesetzt und im Funktionstest nachgewiesen.

Die Kommunikation zwischen Backend und Worker erfolgt über HTTP zwischen zwei eigenständig laufenden Node.js-Prozessen und bildet damit eine einfache verteilte Softwarearchitektur.